TP — Conception d'un pipeline CI sécurisé pour une API Express

Objectif pédagogique

Concevoir et implémenter une chaîne d'intégration continue complète pour une API développée avec **Express.js**, intégrant :

- Contrôle qualité local avant commit
- Intégration continue sur la branche development
- Analyse qualité via **SonarCloud**
- Construction d'une image Docker
- Scan de sécurité via **Trivy**
- Publication vers **GitHub Container Registry**

Contexte

Vous disposez d'un backend Express fonctionnel exposant une API REST.
L'application comporte :

- Un système de lint
- Un formateur de code
- Une suite de tests unitaires

Votre mission consiste à professionnaliser son cycle de livraison en y intégrant une chaîne CI robuste et sécurisée.

Partie 1 — Validation locale avec Git Hooks

Objectif

Mettre en place une validation automatique locale empêchant l'introduction de code non conforme dans le dépôt.

Contraintes

- La validation doit s'exécuter **avant chaque commit**

- Le commit doit être bloqué si :
    - Le lint échoue
    - Le formatage n'est pas conforme

Partie 2 — Mise en place du workflow CI

Déclenchement

Le pipeline doit se déclencher :

- Lors d'un push sur la branche development

Étapes obligatoires du pipeline

Build & Tests

La pipeline doit :

- Installer les dépendances proprement
- Vérifier le lint
- Vérifier le formatage
- Exécuter les tests unitaires
- Générer un rapport de couverture

Le pipeline doit s'arrêter en cas d'échec.

Analyse qualité continue

L'analyse doit être intégrée avec SonarCloud.

Contraintes :

- L'analyse doit être liée au repository GitHub
- La validation doit inclure un Quality Gate
- La pipeline doit être bloquée si le Quality Gate échoue
- La couverture de tests doit être exploitée par l'analyse

Construction d'une image Docker

L'image doit :

- Être construite via un Dockerfile multi-stage
- Être optimisée pour un environnement production
- Exposer uniquement les dépendances nécessaires

- Être taguée intelligemment

Scan de sécurité container

Avant publication :

- Scanner l'image Docker
- Bloquer la pipeline en cas de vulnérabilités critiques ou élevées
- Intégrer l'analyse dans les logs CI

Publication vers GitHub Container Registry

Conditions :

- Publication uniquement si toutes les étapes précédentes réussissent
- Utilisation des permissions GitHub adaptées
- Authentification conforme aux bonnes pratiques GitHub Actions

Contraintes techniques

- Le workflow doit être structuré par jobs dépendants
- Les étapes doivent être conditionnelles
- Les permissions GitHub doivent être correctement configurées
- La solution doit être reproductible
- Les secrets ne doivent jamais être exposés

Livrables attendus

1. Repository GitHub fonctionnel
2. Hooks locaux opérationnels
3. Workflow CI complet et structuré
4. Image visible dans GHCR
5. Projet analysé sur SonarCloud
6. README expliquant l'architecture CI/CD mise en place